

//Minería de Grafos

Presentación de los algoritmos de centralidad

Gael Rendon Mendoza

Hector Alonso Heredia Perez

Paulina Ruiz Uribe

Regina Moch Fuentes



Centralidad

Identificación de
nodos clave y
análisis de
influencia

- Identificar los nodos más importantes
- Evaluar credibilidad y accesibilidad
- Medir la velocidad de distribución de información
- Detectar puentes entre grupos

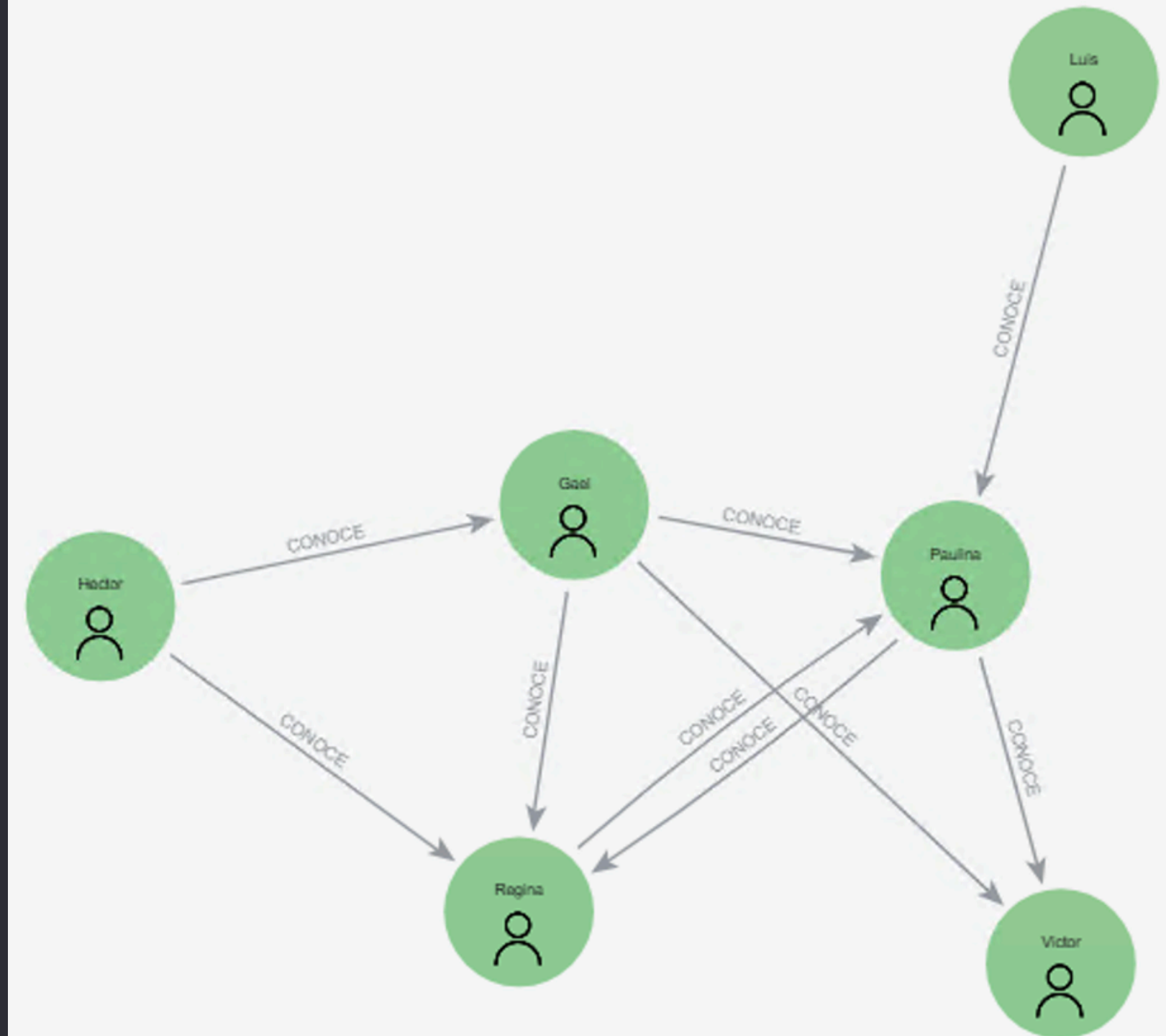
Grafo

```
1 CALL gds.graph.project(  
2   'personasGraph',  
3   'Persona',  
4   {  
5     CONOCE: {  
6       properties: 'weight'  
7     }  
8   }  
9 )  
10 YIELD graphName, nodeCount, relationshipCount;
```

Table RAW

graphName	nodeCount	relationshipCount
-----------	-----------	-------------------

"personasGraph"	6	9
-----------------	---	---



Degree Centrality

Propuesta por Linton C. Freeman en 1979.

Mide el número de conexiones directas que tiene un nodo.

Define el alcance de un nodo con base en su grado.

Grado promedio

Total de relaciones / número total de nodos

Distribución de grado

Probabilidad de que un nodo aleatorio tenga cierto número de relaciones

Obtiene

- Grado mínimo
- Grado máximo
- Grado promedio
- Desviación estándar

- Red social; identificar qué usuario tiene más seguidores directos
- Red organizacional; detectar quién colabora con más departamentos
- Clientes; encontrar al usuario con más conexiones comerciales

Ejemplo

```
CALL gds.degree.stream('personasGraph', {
| relationshipWeightProperty: 'weight'
|})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS persona, score
ORDER BY score DESC;
```

	persona	score
1	"Gael"	2.2
2	"Hector"	1.4
3	"Paulina"	1.35
4	"Regina"	0.75
5	"Luis"	0.3
6	"Victor"	0.0

Closeness Centrality

Indica qué nodo puede alcanzar más fácilmente a todos los demás nodos del grafo

$$C_{norm}(u) = \frac{n-1}{\sum_{v=1}^{n-1} d(u,v)}$$

Promedio de los caminos más cortos, permite comparar grafos de distintos tamaños

- u : nodo analizado
- n : # total de nodos en el grafo
- $d(u,v)$: camino mas corto entre u y v

Nos permite

- Identificar nodos que propagan información más rápido.
- Evaluar la velocidad de interacción cuando las relaciones tienen peso.
- Funciona principalmente en grafos conectados.

- Empresa; detectar que empleado puede comunicar información al resto más rápidamente
- Red de transporte; identificar la estación desde la cual se llega más rápido a todas las demas
- Epidemiología, estimar quién podría propagar una enfermedad con mayor rapidez

Ejemplo

```
CALL gds.closeness.stream('personasGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS persona, score
ORDER BY score DESC;
```

	persona	score
1	"Gael"	1.0
2	"Paulina"	0.8
3	"Regina"	0.8
4	"Victor"	0.625
5	"Hector"	0.0
6	"Luis"	0.0

Betweenness Centrality

Mide el control que tiene un nodo sobre el flujo entre otros nodos o grupos.

Permite detectar cuánta influencia tiene un nodo sobre el flujo de información o recursos.

identifica que nodos o relaciones funcionan como puentes

Nodos pivotaes

Aquellos que están presentes en todos los caminos más cortos entre ciertos nodos.

Si se eliminan, el nuevo camino será más largo

En grafos largos, se utilizan **algoritmos de aproximación** para evitar el costo de un cálculo exacto

- Organización; identificar quien conecta dos áreas que no se comunican directamente
- Red criminal; detectar un intermediario clave entre grupos
- Red de internet; Identificar la caída de que elemento afectaría más la conectividad

Ejemplo

```
CALL gds.betweenness.stream('personasGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS persona, score
ORDER BY score DESC;
```

	persona	score
1	"Paulina"	3.0
2	"Gael"	1.5
3	"Regina"	0.5
4	"Hector"	0.0
5	"Luis"	0.0
6	"Victor"	0.0

PageRank

Determina qué nodo es el más importante dentro de la red
Se basa en la influencia transitiva o direccional.

$$PR(u) = (1 - d) + d \left(\frac{PR(T1)}{C(T1)} + \dots + \frac{PR(Tn)}{C(Tn)} \right)$$

- Una página u recibe enlaces de T1 a Tn.
- d (≈ 0.85): probabilidad de seguir navegando por enlaces.
- 1 - d: probabilidad de llegar directamente a un nodo.
- C(Tn): número de enlaces salientes de T.

Se calcula mediante

- Distribución iterativa del rango de un nodo sobre sus vecinos
- Recorridos aleatorios del grafo contando visitas a cada nodo

Las conexiones hacia nodos más importantes aportan mayor influencia que muchas conexiones a nodos menos relevantes

- Motores de búsqueda;
clasificar páginas web
según su importancia
- Redes sociales; detectar
usuarios influyentes
- Redes académicas;
identificar artículos
altamente citados por
otros artículos
relevantes

Ejemplo

```
CALL gds.pageRank.stream('personasGraph', {
  maxIterations: 20,
  dampingFactor: 0.85,
  relationshipWeightProperty: 'weight'
})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS persona, score
ORDER BY score DESC;
```

	persona	score
1	"Paulina"	0.7921411244151247
2	"Victor"	0.6256031398976238
3	"Regina"	0.5142925269069293
4	"Gael"	0.22285714285714292
5	"Hector"	0.15000000000000002
6	"Luis"	0.15000000000000002

Referencias

Cap. 5 Centrality Algorithms

Graph Algorithms: Practical Examples in Apache Spark and Neo4k

Creemos que esta fue una actividad interesante, y nos dio la oportunidad de aplicar los algoritmos en una base de datos real, aunque pequeña. Los ejemplos del libro fueron muy útiles, y fue interesante saber toda la historia de cada uno de los algoritmos, creemos que eso nos da más visión sobre como funcionan.

